ELEC 475 — Lab 2 Postlab

Christopher Chong - 20340133

Nicolas Rucareanu - 20360005

Architectural Specifications

Base LeNet-5

Layer	Kernel / stride / padding	Output dimensions	Parameters	Receptive field
Input	—	3 × 64 × 64	0	1 × 1
Conv1	5 / 1 / 0	6 × 60 × 60	456	5 × 5
Average Pool1	2 / 2 / 0	6 × 30 × 30	0	6 × 6
Conv2	5 / 1 / 0	16 × 26 × 26	2,416	14 × 14
Average Pool2	2 / 2 / 0	16 × 13 × 13	0	16 × 16
Flatten	—	2,704	0	—
Fully connected 1	2,704 → 120	120	324,600	—
Fully connected 2	120 → 84	84	10,164	—
Output	84 → 10	10	850	—
Total			**338,486**	

Base LeNet-5 processes a 3x64x64 input through two unpadded 5x5 convolutions with 6 and 16 filters, Tanh activations, and 2x2 average pooling with stride 2. The spatial dimensions decrease from 64 → 60 → 30 → 26 → 13, producing 16x13x13=2,704 flattened features for fully connected layers of sizes 120→84→K. For K=10, the model contains 338,486 trainable parameters, and each feature after the second pooling layer has a theoretical receptive field of 16x16 pixels.

Modernized LeNet

Layer	Kernel / stride / padding	Output dimensions	Parameters	Receptive field
Input	—	3 × 64 × 64	0	1 × 1
Conv1	5 / 1 / 2	6 × 64 × 64	456	5 × 5
Max Pool1	3 / 2 / 0	6 × 31 × 31	0	7 × 7
Conv2	5 / 1 / 2	16 × 31 × 31	2,416	15 × 15
Max Pool2	3 / 2 / 0	16 × 15 × 15	0	19 × 19
Flatten	—	3,600	0	—
Fully connected 1	3,600 → 120	120	432,120	—
Fully connected 2	120 → 84	84	10,164	—
Output	84 → 10	10	850	—
Total			**446,006**	

Modernized LeNet replaces Tanh with ReLU and average pooling with overlapping 3x3 max pooling at stride 2, while adding padding 2 to both 5x5 convolutions. Its spatial dimensions follow 64→64→31→31→15, yielding 16x15x15=3,600 flattened features for the 120→84→K classifier. For K=10, the larger first fully connected layer increases the total to 446,006 parameters, while the final pooled features have a theoretical receptive field of 19x19 pixels.

Full AlexNet-64

Layer	Kernel / stride / padding	Output dimensions	Parameters	Receptive field
Input	—	3 × 64 × 64	0	1 × 1
Conv1	5 / 2 / 2	64 × 32 × 32	4,864	5 × 5
Max Pool1	3 / 2 / 0	64 × 15 × 15	0	9 × 9
Conv2	5 / 1 / 2	192 × 15 × 15	307,392	25 × 25
Max Pool2	3 / 2 / 0	192 × 7 × 7	0	33 × 33
Conv3	3 / 1 / 1	384 × 7 × 7	663,936	49 × 49
Conv4	3 / 1 / 1	384 × 7 × 7	1,327,488	65 × 65
Conv5	3 / 1 / 1	256 × 7 × 7	884,992	81 × 81
Max Pool3	3 / 2 / 0	256 × 3 × 3	0	97 × 97
Flatten	—	2,304	0	—
Fully connected 1	2,304 → 1,024	1,024	2,360,320	—
Fully connected 2	1,024 → 1,024	1,024	1,049,600	—
Output	1,024 → 10	10	10,250	—
Total without BN			**6,608,842**	

AlexNet-64 contains five convolutional stages with channel widths 64,192,384,384,256, using a 5x5 first convolution at stride 2, a second 5x5 convolution at stride 1, and three 3x3 convolutions at stride 1, with padding 2 and 1 respectively. ReLU and three overlapping max-pooling layers reduce the input to 256x3x3=2,304 features, followed by a 1,024→1,024→K classifier with dropout probability 0.5. For K=10, the model contains 6,608,842 parameters without normalization or with LRN, and 6,611,402 with BatchNorm, while its final pooled features have a theoretical receptive field of 97x97 pixels.

PyTorch Model Zoo AlexNet

Layer	Kernel / stride / padding	Output dimensions	Parameters	Receptive field
Input	—	3 × 64 × 64	0	1 × 1
Conv1	11 / 4 / 2	64 × 15 × 15	23,296	11 × 11
Max Pool1	3 / 2 / 0	64 × 7 × 7	0	19 × 19
Conv2	5 / 1 / 2	192 × 7 × 7	307,392	51 × 51
Max Pool2	3 / 2 / 0	192 × 3 × 3	0	67 × 67
Conv3	3 / 1 / 1	384 × 3 × 3	663,936	99 × 99
Conv4	3 / 1 / 1	256 × 3 × 3	884,992	131 × 131
Conv5	3 / 1 / 1	256 × 3 × 3	590,080	163 × 163
Max Pool3	3 / 2 / 0	256 × 1 × 1	0	195 × 195
Adaptive Avg Pool	Target: 6 × 6	256 × 6 × 6	0	Same input coverage
Flatten	—	9,216	0	—
Fully connected 1	9,216 → 4,096	4,096	37,752,832	—
Fully connected 2	4,096 → 4,096	4,096	16,781,312	—
Output	4,096 → 10	10	40,970	—
Total			**57,044,810**	

PyTorch's AlexNet uses five convolutional layers with channel widths 64,192,384,256,256, beginning with an 11x11 convolution at stride 4 and followed by kernels of sizes 5,3,3,3. For a 64x64 input, convolution and max pooling produce a 256x1x1 feature map, which adaptive average pooling converts to 256x6x6 before a 9,216→4,096→4,096→K classifier with ReLU and dropout. For K=10, the model contains 57,044,810 parameters, and

the features before adaptive pooling have a theoretical receptive field of 195x195 pixels, including padded regions beyond the image boundaries.

Master table 7.3a: architecture and computational complexity

Model	Conv depth	Normalization & Regularization	Data Augmentation	Trainable Params (M)	FLOPs (MFLOPs)	Mean Epoch Time (s)	Inference Latency (ms)
Base LeNet-5	2	None	No	0.338	7.17	59.66	0.39
Modernized LeNet	2	None (ReLU + overlapping max pooling)	No	0.446	9.19	66.78	0.44
AlexNet (no norm)	5	Dropout (p=0.5)	No	6.609	436.70	45.34	2.66
AlexNet + LRN	5	LRN + Dropout (p=0.5)	No	6.609	436.70	52.47	4.71
AlexNet + BN	5	BatchNorm2d + Dropout (p=0.5)	No	6.611	437.97	48.93	3.11
AlexNet + BN + Aug	5	BatchNorm + Dropout (p=0.5)	Yes	6.611	437.97	49.44	3.11
PyTorch Zoo AlexNet	5	PyTorch Zoo (AdaptiveAvgPool2d)	No	57.045	188.20	40.89	8.73

Adding LRN increased mean epoch time from 45.34 s to 52.47 s (15.7%), while BatchNorm increased it to 48.93s (7.9%) and increased reported FLOPs from 436.70 to 437.97 MFLOPs (0.3%). The LRN and no-normalization models both report 436.70 MFLOPs because the profiler does not count LRN operations, although those operations still add computational work. BatchNorm widens the range of learning rates that train without diverging, it makes the effective step size self-correct as weights grow, and it keeps gradient magnitudes in a stable range across depth. LeNet models were trained on CPU, while AlexNet models were trained on GPU. Their epoch times therefore reflect both architectural and hardware differences and should not be compared as a measure of model efficiency alone.

Master table 7.3b: final-epoch predictive performance

Model	CIFAR-10 Val Acc %	CIFAR-10 Val Macro F1 %	Tiny ImageNet ValAcc %	Tiny ImageNet Val Macro F1 %
Base LeNet-5	52.98	53.12	10.12	7.97
Modernized LeNet	64.51	64.00	18.83	16.70
AlexNet (no norm)	74.62	74.38	0.50	0.00
AlexNet + LRN	72.82	72.64	16.85	14.40
AlexNet + BN	77.54	76.96	32.70	31.10
AlexNet + BN + augmentation	80.96	81.03	34.48	32.97
Pytorch Zoo AlexNet	67.37	67.19	0.50	0.00

BatchNorm provided the better trade-off in these runs, achieving higher validation accuracy and Macro F1 than both alternatives with less runtime than LRN. BatchNorm changes the geometry of the loss surface and the scale-dependence of the gradients, so optimization becomes better conditioned. Normalizing mini-batch activations fixes the mean and variance of each layer's inputs, so downstream layers see a more stable distribution. It also bounds how much the loss and its gradient can change per step, which smooths the landscape.

Extended ablation exploration and generalization gap

Condition	Seeds	Train Acc mean ± std	Val Acc mean ± std	Gap mean ± std pp
alexnet_bn	3	90.58 ± 0.08	76.89 ± 1.22	13.69 ± 1.26
alexnet_bn_aug	3	84.68 ± 0.28	81.54 ± 0.65	3.15 ± 0.37
alexnet_bn_aug_lr0003	3	87.18 ± 0.19	81.97 ± 1.07	5.21 ± 0.95

Multi-seed normalization and augmentation results

Configuration	Epoch	Seeds completed	Acc mean ± std %	Macro F1 mean ± std %
AlexNet (no norm)	10	3	74.43 ± 0.35	74.28 ± 0.22
AlexNet + LRN	10	3	72.82 ± 2.15	72.71 ± 1.99
AlexNet + BN	10	3	78.30 ± 0.46	78.17 ± 0.95
AlexNet + BN + augmentation	15	3	81.54 ± 0.65	81.43 ± 0.48

Technical analysis and discussion

The base LeNet achieved final validation accuracy of 52.98% and Macro F1 of 53.12%. Modernized LeNet reached 64.51% and 64.00%, improvements of 11.53 and 10.88 percentage points**.** Modernized LeNet's best checkpoint was epoch 4, while base LeNet's was epoch 5. Their final training–validation accuracy gaps were 4.27 and 4.74 percentage points, respectively. Modernized LeNet used 31.8% more parameters and 28.1% more profiler FLOPs, with mean epoch time increasing from 59.66 to 66.78 seconds. ReLU and overlapping max pooling may have contributed to the improvement, but padding and classifier capacity also changed, preventing attribution to any single modification.

At epoch 10, BatchNorm achieved 78.73% validation accuracy and 79.11% Macro F1, compared with 74.62% and 74.38% without normalization, and 72.82% and 72.64% with LRN. BatchNorm therefore improved accuracy by 4.11 percentage points over no normalization and 5.91 points over LRN. Standardizing activations and applying learned scaling may have improved optimization under these settings, but the experiments do not establish guaranteed bounds on gradient changes or protection against divergence. Across three seeds, BatchNorm also achieved the highest mean accuracy, supporting the benefit beyond one initialization.

Normalization and computational overhead: LRN increased mean epoch time from 45.34 to 52.47 seconds, while BatchNorm increased it to 48.93 seconds and reported FLOPs from 436.70 to 437.97 MFLOPs. The profiler excludes LRN calculations, explaining its identical reported FLOPs despite longer execution time. BatchNorm offered the better accuracy–runtime trade-off in these runs.

Convergence stability: All three training-loss curves decreased smoothly. BatchNorm reached the lowest ten-epoch loss, 0.437, compared with 0.679 without normalization and 0.672 with LRN. **BatchNorm showed slightly more consistent training-loss reductions:** the standard deviation of successive loss changes was 0.109, versus 0.119 and 0.117, respectively. However, its validation loss fluctuated more. Gradients were not tracked throughout these training runs, so this supports consistency in training-loss reduction rather than proving greater gradient stability.

Augmentation and generalization gap: At epoch 15, augmentation increased BatchNorm's CIFAR-10 validation accuracy from 77.54% to 80.96% and Macro F1 from 76.96% to 81.03%. Training accuracy decreased from 90.62% to 84.45%, reducing the training–validation gap from 13.08 to 3.49 percentage points. This suggests better generalization, although augmented training images make the gaps less directly comparable. Across three seeds, mean validation accuracy increased from 76.89% to 81.54%. Random crops, flips, and colour changes likely encouraged features that remain useful across image variations. These gains added no model parameters or inference FLOPs, while mean epoch time increased slightly from 48.93 to 49.44 seconds.

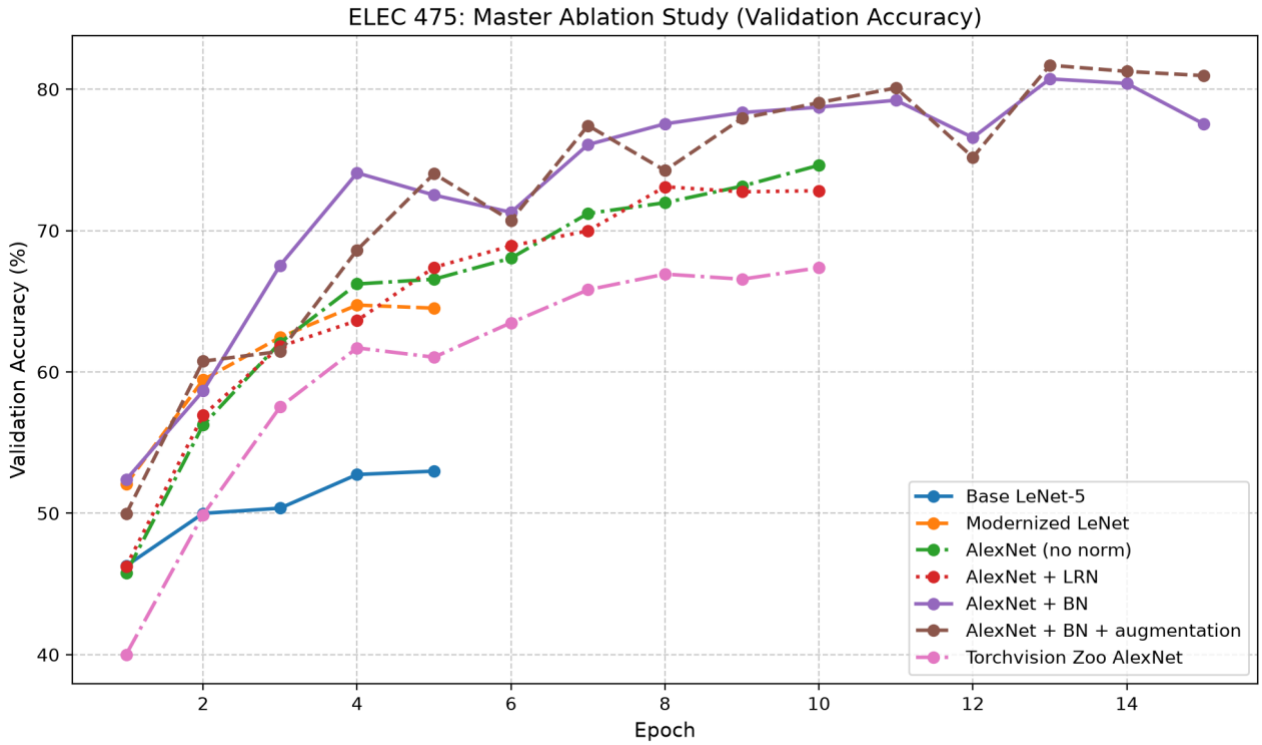
Tiny ImageNet performance: Tiny ImageNet contains 200 classes with 500 training images each, compared with ten classes and 5,000 images each for CIFAR-10. More similar categories and fewer examples per class make classification harder. BatchNorm achieved 32.70% accuracy and 31.10% Macro F1, well above the 0.5% chance baseline but below its CIFAR-10 results of 77.54% and 76.96%. Augmentation improved Tiny ImageNet performance to 34.48% accuracy and 32.97% Macro F1.

Overfitting trajectory: Tiny ImageNet did **not** show earlier validation-loss deterioration than CIFAR-10. Its BatchNorm validation loss generally decreased through epoch 14 before rising at epoch 15, whereas CIFAR-10 already showed temporary increases at epochs 5–6. The final Tiny ImageNet increase, while training loss continued decreasing, suggests late overfitting. The original no-normalization run at learning rate 0.001 instead remained at chance-level

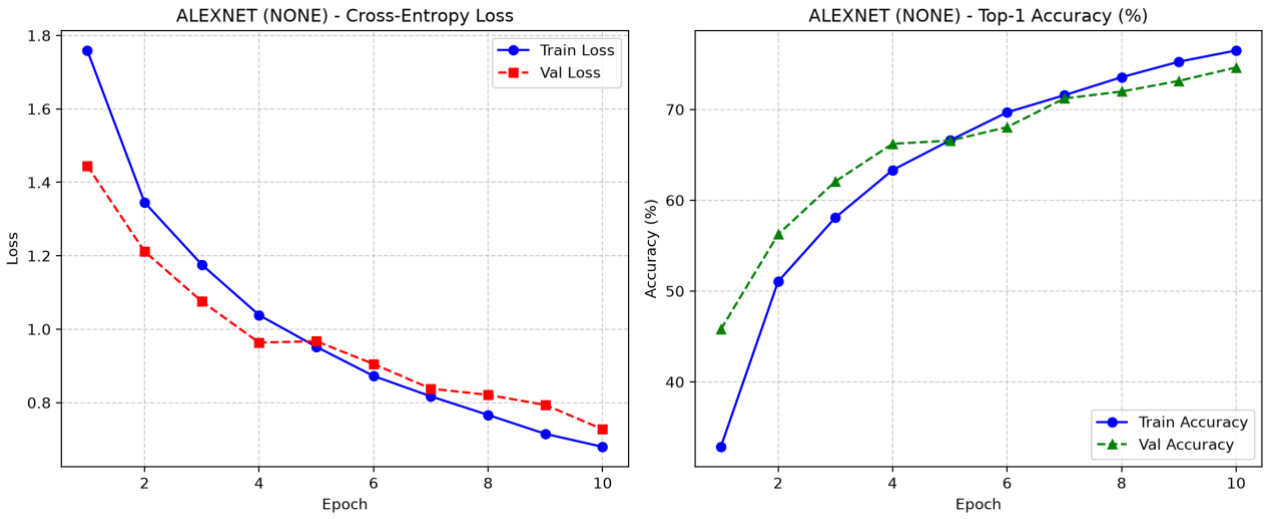
training and validation performance, indicating failed optimization. The separate 0.0001 retry reached 24.24% accuracy and 21.68% Macro F1, demonstrating that the failure depended on the training settings.

PyTorch Zoo comparison: Zoo AlexNet achieved 67.37% accuracy and 67.19% Macro F1 on CIFAR-10, below custom AlexNet without normalization despite having considerably more parameters. Its aggressive downsampling and larger classifier differ from AlexNet-64, so greater parameter count does not guarantee better performance. On Tiny ImageNet, Zoo AlexNet remained at 0.50% accuracy, indicating unsuccessful learning under the chosen settings; the specific cause was not established.

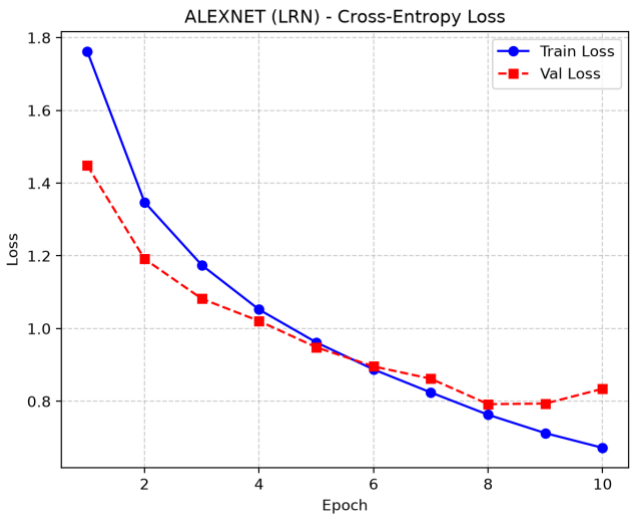
Figures

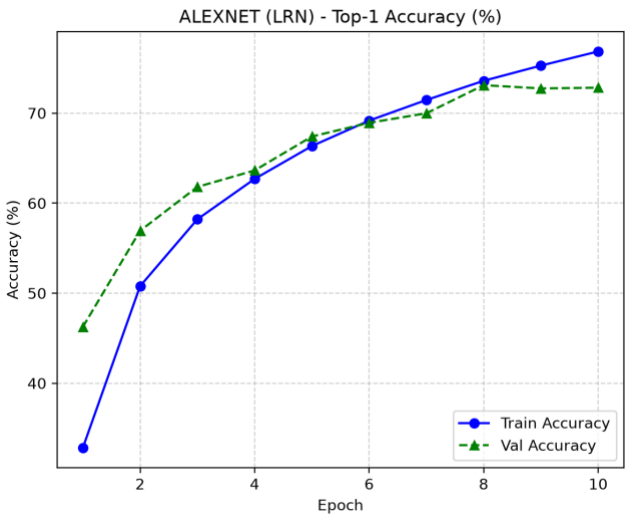


Completed CIFAR-10 ablation trajectories

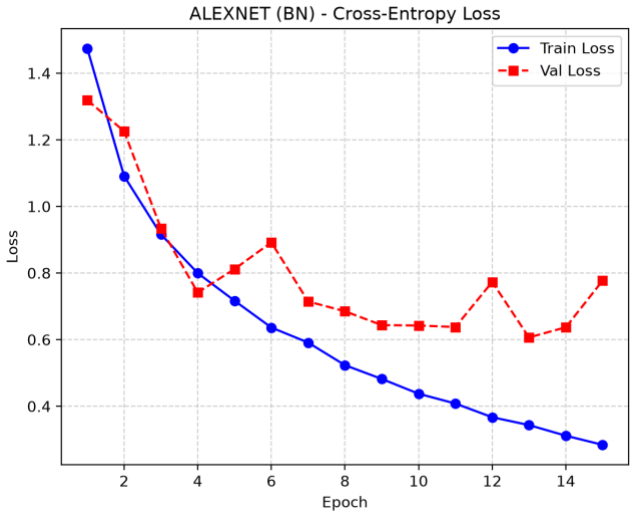


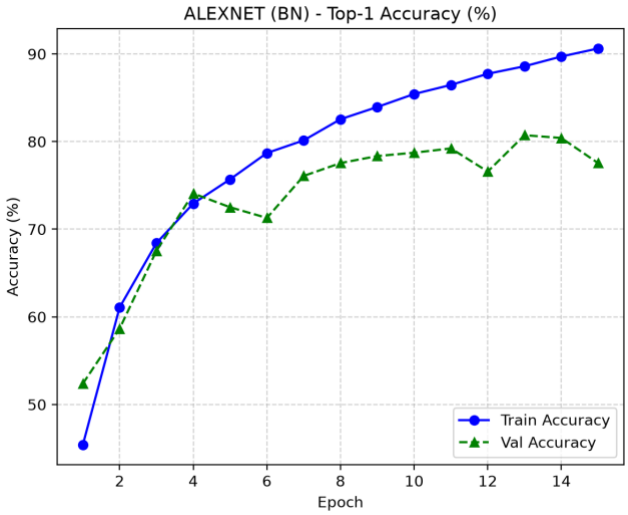
alexnet_none: training and validation curves



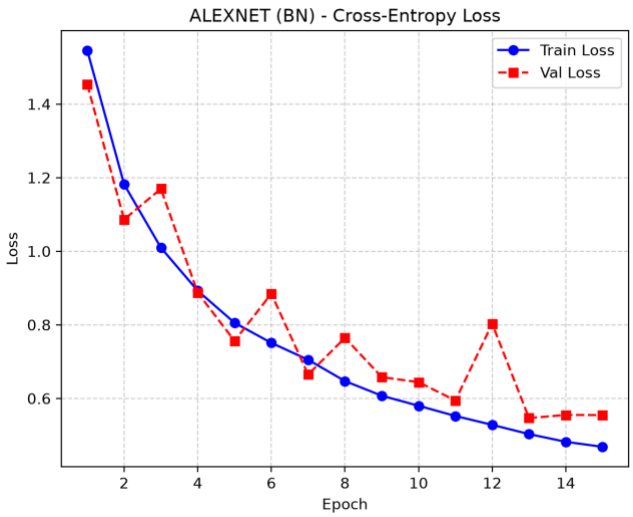


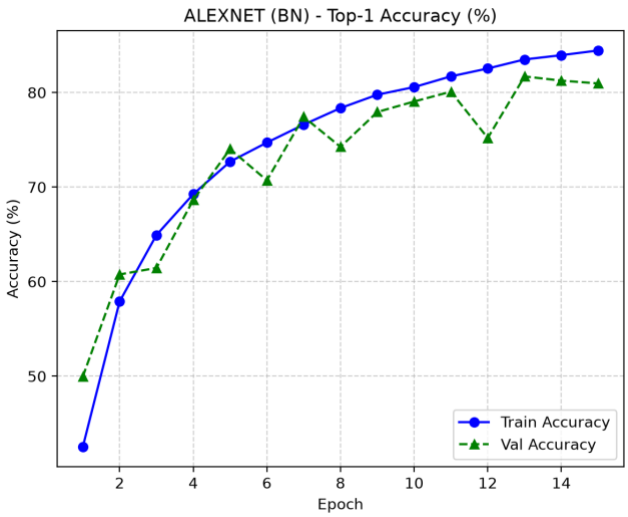
alexnet_LRN: training and validation curves



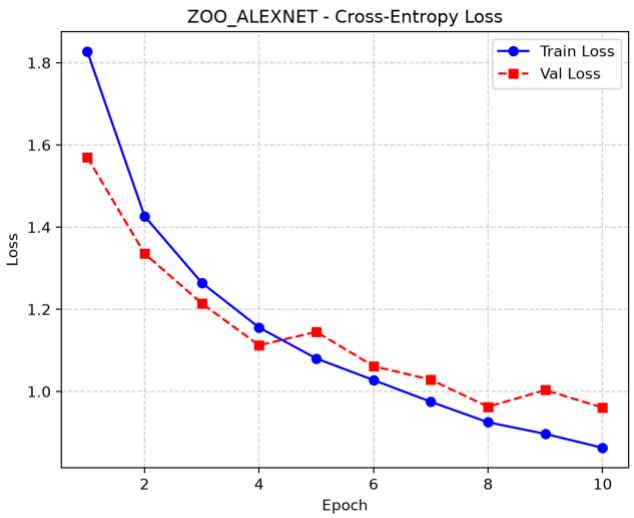


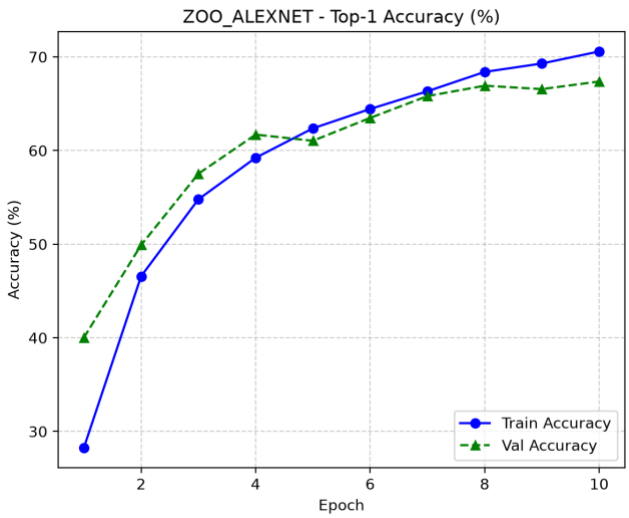
alexnet_BN: training and validation curves



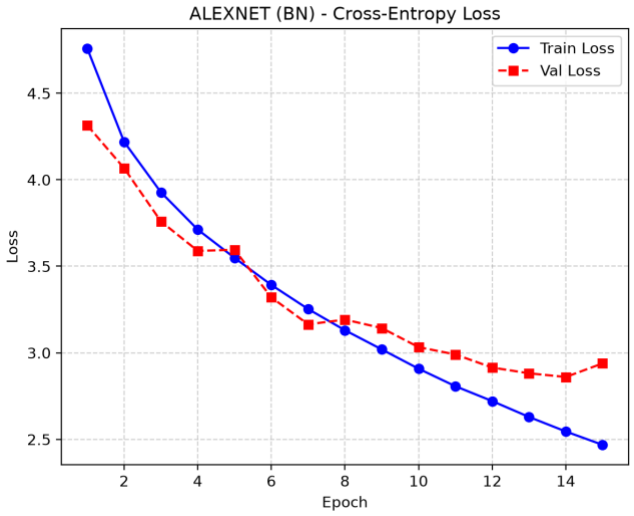


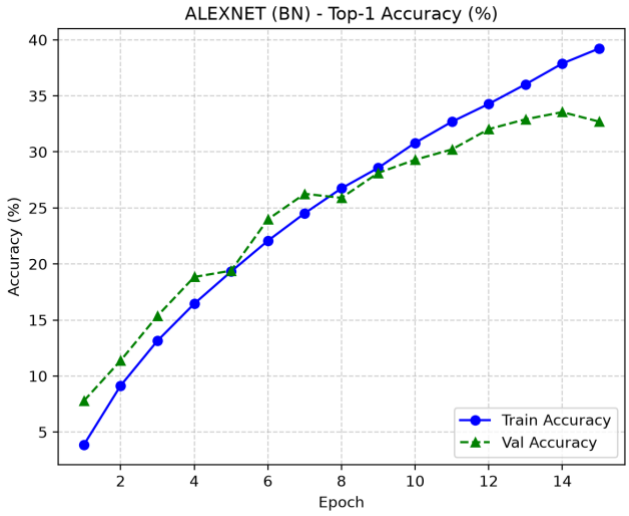
alexnet_BN_aug: training and validation curves





zoo_alexnet: training and validation curves





Tiny ImageNet BN curves

```
(.venv) (base) christopherchong@Christophers-MacBook-Pro lab2_template % cat logs/postlab-transforms.log
================================================================================
ELEC 475: Transform Paradigm & Variable Type Diagnostic
================================================================================

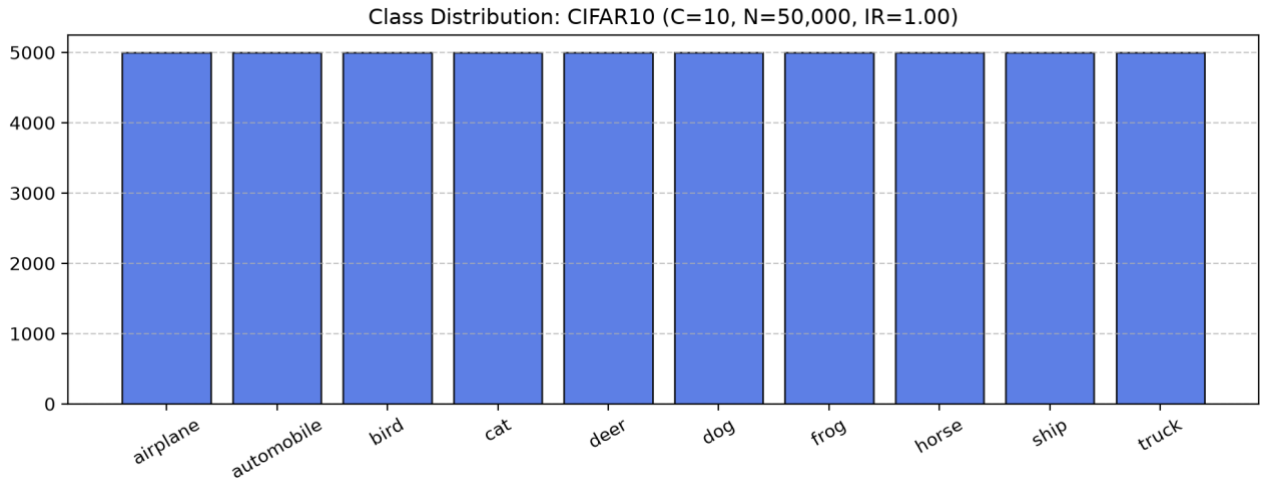
[1] OpenCV (cv2) Output:
    - Object Type      : <class 'numpy.ndarray'>
    - Data Type (dtype): float32
    - Shape (Layout)   : (64, 64, 3)  --> [Height, Width, Channels] (Channels-Last)
    - Value Range      : Min = 0.0000, Max = 1.0000
    - Color Space      : Converted BGR -> RGB
    - Memory Location  : Host System RAM (CPU)

[2] NumPy Output:
    - Object Type      : <class 'numpy.ndarray'>
    - Data Type (dtype): float32
    - Shape (Layout)   : (64, 64, 3)  --> [Height, Width, Channels] (Channels-Last)
    - Value Range      : Min = 0.0000, Max = 1.0000
    - Memory Location  : Host System RAM (CPU)

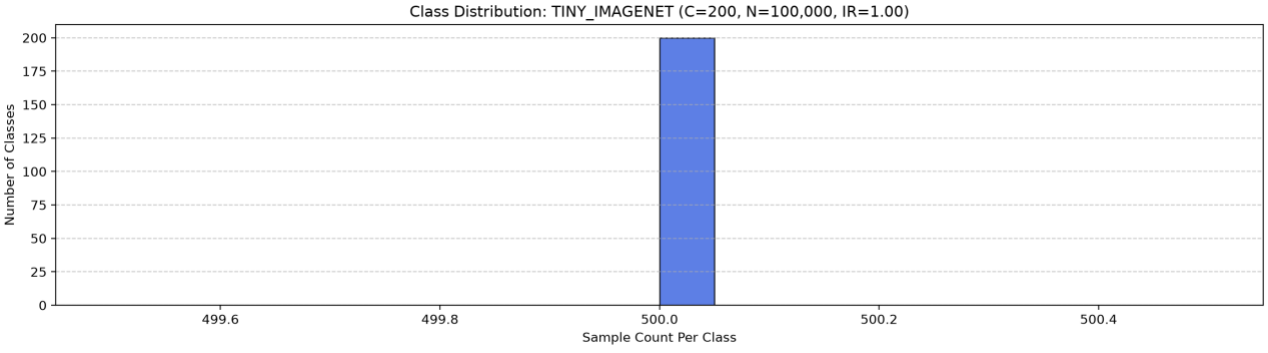
[3] PyTorch (torchvision) Tensor Output:
    - Object Type      : <class 'torch.Tensor'>
    - Data Type (dtype): torch.float32
    - Shape (Layout)   : torch.Size([3, 64, 64])  --> [Channels, Height, Width] (Channels-First)
    - Value Range      : Min = -1.9895, Max = 2.1265 (Standardized)
    - Memory Location  : cpu (Host CPU RAM)

--------------------------------------------------------------------------------
Benchmark: Processing 1,000 Images (Resize + Flip + Normalize)
--------------------------------------------------------------------------------
- OpenCV (CPU)            :    6.80 ms  (147,089 img/s) + Host-to-Device Copy required
- NumPy (CPU)             :   14.90 ms  ( 67,096 img/s) + Host-to-Device Copy required
- PyTorch (CPU Tensors)   :   97.56 ms  ( 10,250 img/s)
```

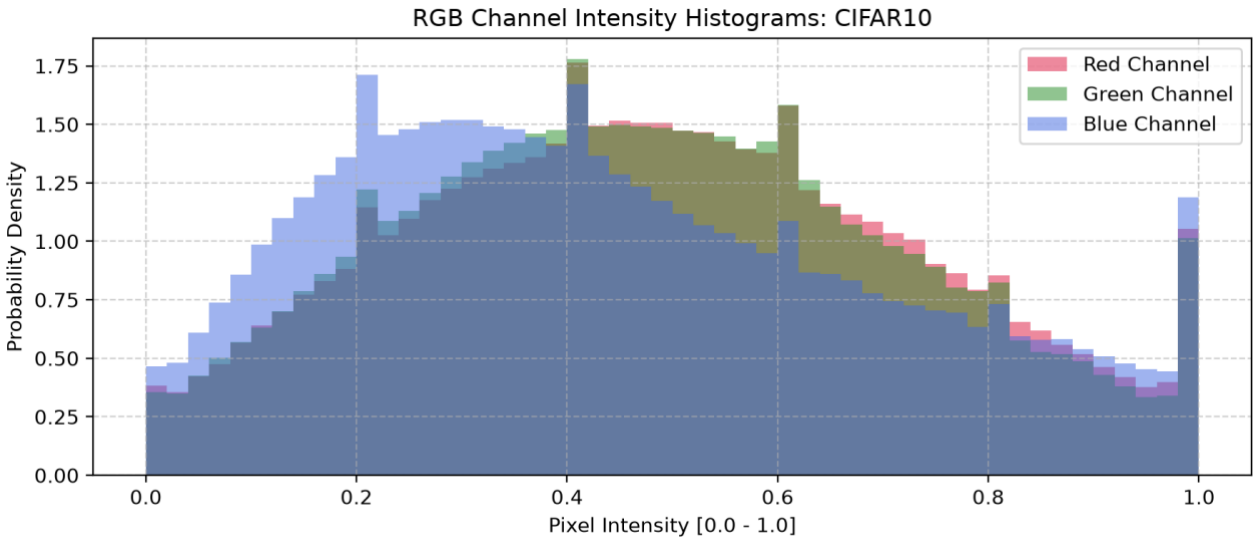
Sample output from demo_transforms.py



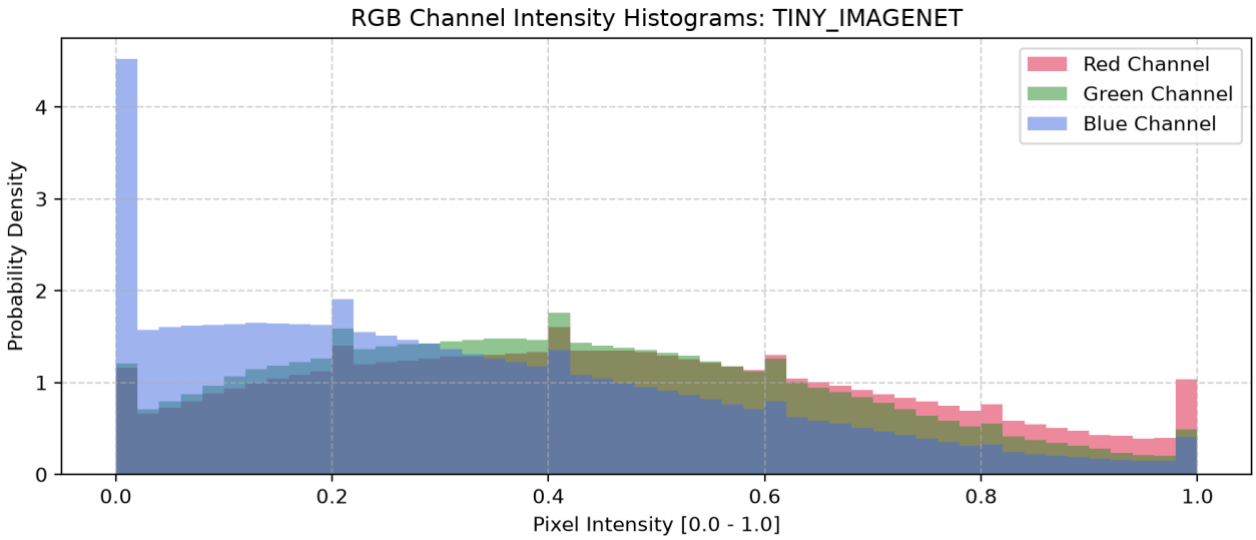
eda cifar10 class distribution



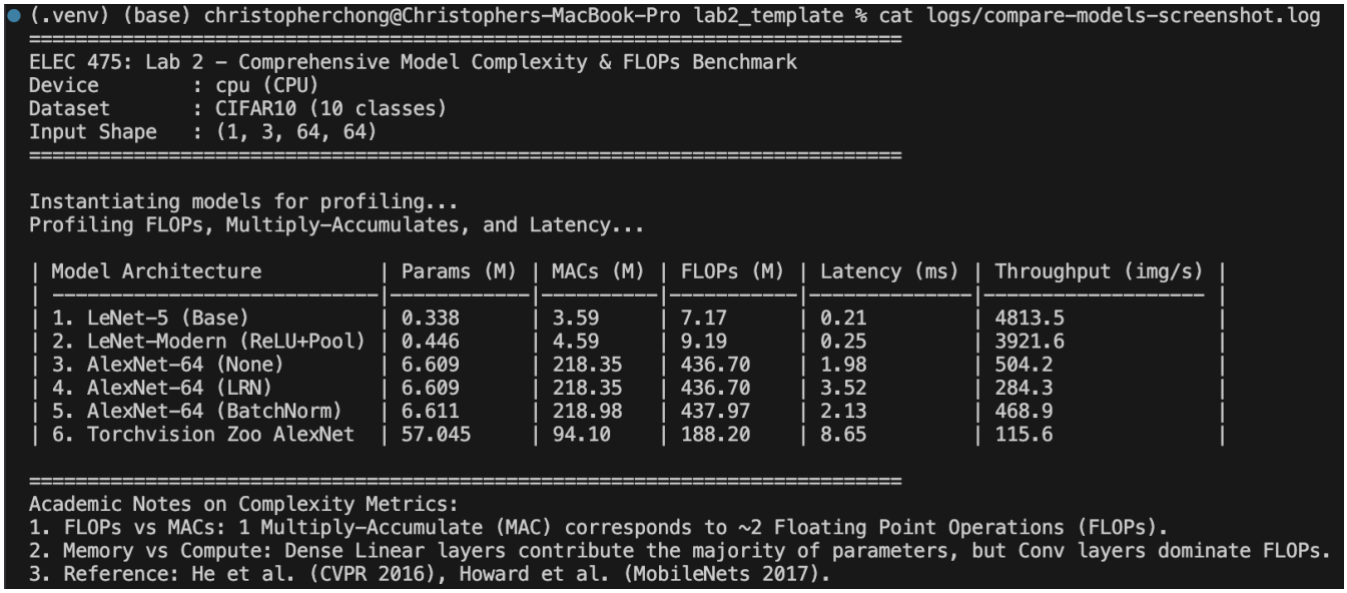
eda tiny imagenet class distribution



eda cifar10 channel histograms



eda tiny imagenet channel histograms



```
(.venv) (base) christopherchong@Christophers-MacBook-Pro lab2_template % cat logs/compare-models-screenshot.log
================================================================================
ELEC 475: Lab 2 - Comprehensive Model Complexity & FLOPs Benchmark
Device        : cpu (CPU)
Dataset       : CIFAR10 (10 classes)
Input Shape   : (1, 3, 64, 64)
================================================================================

Instantiating models for profiling...
Profiling FLOPs, Multiply-Accumulates, and Latency...

| Model Architecture          | Params (M) | MACs (M) | FLOPs (M) | Latency (ms) | Throughput (img/s) |
| --------------------------- |----------- |--------- |---------- |------------- |------------------- |
| 1. LeNet-5 (Base)           | 0.338      | 3.59     | 7.17      | 0.21         | 4813.5             |
| 2. LeNet-Modern (ReLU+Pool) | 0.446      | 4.59     | 9.19      | 0.25         | 3921.6             |
| 3. AlexNet-64 (None)        | 6.609      | 218.35   | 436.70    | 1.98         | 504.2              |
| 4. AlexNet-64 (LRN)         | 6.609      | 218.35   | 436.70    | 3.52         | 284.3              |
| 5. AlexNet-64 (BatchNorm)   | 6.611      | 218.98   | 437.97    | 2.13         | 468.9              |
| 6. Torchvision Zoo AlexNet  | 57.045     | 94.10    | 188.20    | 8.65         | 115.6              |

================================================================================
Academic Notes on Complexity Metrics:
1. FLOPs vs MACs: 1 Multiply-Accumulate (MAC) corresponds to ~2 Floating Point Operations (FLOPs).
2. Memory vs Compute: Dense Linear layers contribute the majority of parameters, but Conv layers dominate FLOPs.
3. Reference: He et al. (CVPR 2016), Howard et al. (MobileNets 2017).
```

Output table from compare_models.py